

CI/CD Pipeline - 50 Interview Questions with Answers

CI/CD Fundamentals (Questions 1-10)

Q1. What is CI/CD?

Answer: CI (Continuous Integration): Practice of automatically integrating code changes from multiple contributors into a shared repository frequently (multiple times per day).

CD (Continuous Delivery/Deployment):

- **Continuous Delivery:** Automatically prepare code for release (can deploy anytime)
- **Continuous Deployment:** Automatically deploy every change to production

Benefits:

- Faster feedback
- Reduced integration problems
- Higher quality code
- Faster time to market
- Automated testing and deployment

Q2. Explain the typical CI/CD pipeline stages.

Answer:

1. Source/Code Commit:

- Developer pushes code to version control (Git)
- Triggers the pipeline

2. Build:

- Compile code
- Resolve dependencies
- Create artifacts

3. Test:

- Unit tests
- Integration tests
- Security scans

4. Package:

- Create Docker images
- Package artifacts
- Version tagging

5. Deploy to Staging:

- Deploy to test environment
- Run smoke tests

6. Deploy to Production:

- Blue-green or canary deployment
- Health checks
- Rollback capability

7. Monitor:

- Application monitoring
- Log aggregation
- Alerts

Q3. What is the difference between Continuous Delivery and Continuous Deployment?

Answer:

Continuous Delivery:

- Automated pipeline up to production-ready state
- **Manual approval** required for production deployment
- Can deploy anytime with one click
- Lower risk, more control

Continuous Deployment:

- Fully automated to production
- **No manual intervention**
- Every change that passes tests goes to production
- Fastest delivery, requires high confidence in testing

Example:

- Continuous Delivery: Release every 2 weeks after manual approval
- Continuous Deployment: 50+ deployments per day automatically

Q4. What are the key principles of CI/CD?

Answer:

1. Automate Everything:

- Build, test, deploy, infrastructure

2. Keep the Build Fast:

- Developers need quick feedback (<10 minutes)

3. Test in Production-like Environment:

- Staging should mirror production

4. Everyone Commits Daily:

- Integrate frequently to avoid merge hell

5. Fix Broken Builds Immediately:

- Broken build is top priority

6. Keep Build Artifacts:

- Versioned, immutable artifacts

7. Deploy the Same Way Everywhere:

- Same process for all environments

8. Make it Easy to Rollback:

- Quick revert capability

Q5. What is a build artifact?

Answer:

Build Artifact: Immutable output of the build process.

Examples:

- JAR/WAR files (Java)
- Docker images
- NPM packages
- Python wheels
- Compiled binaries
- ZIP archives

Characteristics:

- **Versioned:** Semantic versioning (1.2.3)
- **Immutable:** Never changes once created
- **Stored:** Artifact repository (Nexus, Artifactory, ECR)
- **Reusable:** Same artifact deployed to all environments

Best practices:

- Build once, deploy many times
- Include metadata (commit hash, build number)
- Store in centralized repository
- Retention policy for old artifacts

Q6. What is version control and why is it important for CI/CD?

Answer:

Version Control: System to track changes to code over time.

Popular systems:

- **Git:** Distributed (GitHub, GitLab, Bitbucket)
- **SVN:** Centralized (legacy)
- **Mercurial:** Distributed

Importance for CI/CD:

- **Trigger:** Code commit triggers pipeline
- **Traceability:** Know who changed what and when
- **Rollback:** Revert to previous versions
- **Branching:** Parallel development
- **Collaboration:** Multiple developers work together
- **History:** Complete audit trail

Git workflow:

- Feature branches
- Pull requests
- Code reviews
- Merge to main
- Trigger CI/CD

Q7. Explain trunk-based development vs feature branch workflow.

Answer:

Trunk-Based Development:

- All developers commit to main/trunk daily
- Short-lived feature branches (<1 day)
- Feature flags for incomplete features
- Requires discipline and good tests

Pros: Fast integration, simpler, true CI **Cons:** Requires maturity, risk of breaking main

Feature Branch Workflow (GitFlow):

- Long-lived feature branches
- Pull requests for code review
- Merge when feature complete
- Main branch always stable

Pros: Safe, code review process **Cons:** Integration hell, delayed feedback

Best practice: Short-lived branches (1-3 days) with frequent merges.

Q8. What is a pipeline trigger?

Answer:

Pipeline Trigger: Event that starts CI/CD pipeline.

Types:

1. Push/Commit Trigger:

- Most common
- On git push to specific branch

```
on:
  push:
    branches: [main, develop]
```

2. Pull Request Trigger:

- Run tests on PR
- Block merge if tests fail

3. Schedule Trigger (Cron):

- Nightly builds
- Periodic security scans

```
on:
  schedule:
    - cron: '0 2 * * *' # 2 AM daily
```

4. Manual Trigger:

- Operator initiates
- Production deployments

5. Tag Trigger:

- On version tag
- Release process

6. Webhook Trigger:

- External service triggers
- Dependency updates

Q9. What are environment variables in CI/CD?

Answer:

Environment Variables: Configuration values passed to pipeline and application.

Types:

1. Build-time:

- Build number
- Git commit hash
- Branch name

2. Configuration:

- API endpoints
- Database URLs
- Feature flags

3. Secrets:

- API keys
- Passwords
- Certificates

Best practices:

- Never hardcode secrets
- Use secret management (Vault, AWS Secrets Manager)
- Different values per environment
- Inject at runtime

Example (GitHub Actions):

```
env:  
  NODE_ENV: production  
  DATABASE_URL: ${ secrets.DB_URL }
```

Q10. What is infrastructure as code (IaC) in CI/CD context?

Answer:

Infrastructure as Code: Managing infrastructure using code/configuration files.

Tools:

- **Terraform:** Multi-cloud
- **CloudFormation:** AWS
- **Ansible:** Configuration management
- **Pulumi:** Programming languages
- **Kubernetes:** Container orchestration

Benefits for CI/CD:

- **Version controlled:** Track infrastructure changes
- **Automated:** Provision via pipeline
- **Consistent:** Same infrastructure everywhere

- **Reproducible:** Spin up identical environments
- **Disaster recovery:** Quick rebuild

Example (Terraform in pipeline):

```
- name: Terraform Apply
  run: |
    terraform init
    terraform plan
    terraform apply -auto-approve
```

CI/CD Tools (Questions 11-20)

Q11. Compare popular CI/CD tools: Jenkins, GitLab CI, GitHub Actions, CircleCI.

Answer:

Feature	Jenkins	GitLab CI	GitHub Actions	CircleCI
Type	Self-hosted	Integrated	Cloud	Cloud/Self-hosted
Setup	Complex	Easy	Easy	Easy
Cost	Free (hosting cost)	Free tier	Free tier	Free tier
Config	Groovy/UI	YAML	YAML	YAML
Plugins	1000+	Built-in	Marketplace	Orbs
Learning Curve	Steep	Moderate	Easy	Easy

When to use:

- **Jenkins:** Enterprise, complex workflows, existing investment
- **GitLab CI:** Using GitLab, all-in-one platform
- **GitHub Actions:** Using GitHub, simple to moderate pipelines
- **CircleCI:** Fast builds, good caching

Q12. What is Jenkins and its architecture?

Answer:

Jenkins: Open-source automation server for CI/CD.

Architecture:

1. Master/Controller:

- Schedules jobs
- Monitors agents
- Serves UI
- Stores configurations

2. Agents/Nodes:

- Execute jobs
- Can be containerized
- Different OS/environments

3. Jobs/Pipelines:

- Define what to do
- Groovy DSL or YAML

Types:

- **Freestyle:** GUI-based
- **Pipeline:** Code-based (Jenkinsfile)

Jenkinsfile example:

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'make'
      }
    }
    stage('Test') {
      steps {
        sh 'make test'
      }
    }
  }
}
```

Q13. Explain GitHub Actions workflow syntax.

Answer:

GitHub Actions: CI/CD platform integrated with GitHub.

Key concepts:

- **Workflow:** Automated process (YAML file)
- **Event:** Trigger (push, PR, schedule)
- **Job:** Set of steps
- **Step:** Individual task
- **Action:** Reusable unit
- **Runner:** Server that executes jobs

Example workflow:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test

      - name: Build
        run: npm run build
```

Features:

- Matrix builds (multiple versions)
- Caching
- Artifacts
- Secrets management
- Marketplace actions

Q14. What is GitLab CI/CD and .gitlab-ci.yml?

Answer:

GitLab CI/CD: Built-in CI/CD in GitLab.

Components:

- **Pipeline:** Workflow
- **Stages:** Sequential groups (build, test, deploy)
- **Jobs:** Tasks within stages
- **Runners:** Execute jobs (shared or self-hosted)

.gitlab-ci.yml example:

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - echo "Building..."
    - npm install
    - npm run build
  artifacts:
    paths:
      - dist/

test_job:
  stage: test
  script:
    - npm test
  coverage: '/Coverage: \d+\.\d+/'

deploy_job:
  stage: deploy
  script:
    - echo "Deploying..."
    - ./deploy.sh
  only:
    - main
  when: manual
```

Features:

- Auto DevOps
- Built-in container registry
- Security scanning
- Review apps

Q15. What is Docker and its role in CI/CD?

Answer:

Docker: Containerization platform.

Container: Lightweight, standalone package with code and dependencies.

Role in CI/CD:**1. Consistent Environments:**

- Dev, test, prod use same container
- "Works on my machine" solved

2. Build Artifacts:

- Docker images as artifacts
- Versioned and immutable

3. Isolation:

- Each service in own container
- Microservices architecture

4. Pipeline Execution:

- Run builds in containers
- Clean environment each time

5. Easy Deployment:

- Deploy container to any platform
- Kubernetes, ECS, Docker Swarm

Dockerfile example:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

In pipeline:

```
- name: Build Docker image
  run: docker build -t myapp:${{ github.sha }} .

- name: Push to registry
  run: docker push myapp:${{ github.sha }}
```

Q16. What is Kubernetes in CI/CD context?

Answer:

Kubernetes (K8s): Container orchestration platform.

Key concepts:

- **Pod:** Smallest deployable unit (1+ containers)
- **Deployment:** Manages pod replicas
- **Service:** Network access to pods

- **Ingress:** External HTTP access
- **ConfigMap:** Configuration
- **Secret:** Sensitive data

Role in CD:

1. Deployment Target:

- Deploy containers to K8s cluster
- Rolling updates
- Health checks

2. Scaling:

- Horizontal pod autoscaling
- Handle traffic spikes

3. Self-healing:

- Restart failed pods
- Replace unhealthy nodes

4. Declarative Config:

- YAML manifests
- Version controlled

Deployment in pipeline:

```
- name: Deploy to K8s
  run: |
    kubectl set image deployment/myapp \
      myapp=myapp:${{ github.sha }}
    kubectl rollout status deployment/myapp
```

Tools:

- **Helm:** Package manager
- **ArgoCD:** GitOps CD
- **Flux:** GitOps CD

Q17. What is artifact repository and why is it needed?

Answer:

Artifact Repository: Centralized storage for build artifacts.

Popular tools:

- **JFrog Artifactory:** Universal
- **Nexus Repository:** Java-focused

- **Docker Hub/ECR/GCR:** Container images
- **NPM/PyPI:** Language-specific

Purpose:**1. Storage:**

- Store versioned artifacts
- Central location

2. Distribution:

- Fast artifact retrieval
- Caching

3. Security:

- Scan for vulnerabilities
- Access control

4. Compliance:

- Audit trail
- License management

5. Promotion:

- Promote artifacts between environments
- Dev → QA → Prod

Workflow:

Build → Test → Publish to Artifactory → Deploy from Artifactory

Benefits:

- Don't rebuild for each environment
- Reproducibility
- Faster deployments

Q18. Explain blue-green deployment.

Answer:

Blue-Green Deployment: Run two identical production environments.

Process:

1. **Blue:** Current production (v1.0)
2. **Green:** New version (v2.0)
3. Deploy to Green

4. Test Green thoroughly
5. Switch traffic from Blue to Green
6. Keep Blue as rollback option

Benefits:

- **Zero downtime:** Instant switch
- **Easy rollback:** Switch back to Blue
- **Full testing:** Test in production environment

Drawbacks:

- **Cost:** Double infrastructure
- **Database:** Schema changes complex
- **Stateful apps:** Session management

Implementation:

- Load balancer switches traffic
- DNS update
- Kubernetes service selector

Example (Kubernetes):

```
# Service points to blue
selector:
  app: myapp
  version: blue

# After testing, update to:
selector:
  app: myapp
  version: green
```

Q19. What is canary deployment?**Answer:**

Canary Deployment: Gradually roll out to subset of users.

Process:

1. Deploy new version to small % (5%)
2. Monitor metrics (errors, latency)
3. If good, increase to 25%
4. Continue increasing: 50%, 75%, 100%
5. If issues, rollback immediately

Benefits:

- **Lower risk:** Only affects small group

- **Real user feedback:** Production traffic
- **Gradual rollout:** Control blast radius

Monitoring:

- Error rates
- Latency (p50, p95, p99)
- Business metrics
- User feedback

Tools:

- **Flagger:** Kubernetes progressive delivery
- **AWS App Mesh:** Service mesh
- **Istio:** Traffic management

Example (Istio):

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
spec:
  http:
    - route:
        - destination:
            host: myapp
            subset: v1
            weight: 90
        - destination:
            host: myapp
            subset: v2
            weight: 10
```

Q20. What is rolling deployment?**Answer:**

Rolling Deployment: Gradually replace old version with new.

Process (e.g., 10 instances):

1. Start: All 10 running v1
2. Stop 2 instances
3. Start 2 instances with v2
4. Repeat until all are v2

Benefits:

- **No downtime:** Always some instances running
- **Resource efficient:** Don't need double infrastructure
- **Gradual:** Catch issues early

Drawbacks:

- **Slower:** Takes time to roll out
- **Mixed versions:** Both versions running simultaneously
- **Harder rollback:** Need to roll back

Kubernetes rolling update:

```

apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2      # Max new pods beyond desired
      maxUnavailable: 2 # Max pods that can be unavailable

```

Control:

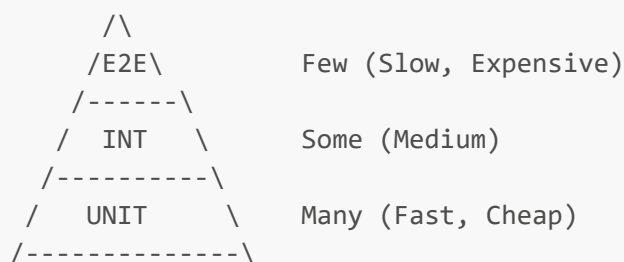
- **maxSurge:** How many extra pods during update
- **maxUnavailable:** How many can be down
- **minReadySeconds:** Wait time before considering pod ready

Testing in CI/CD (Questions 21-30)

Q21. Explain the testing pyramid in CI/CD.

Answer:

Testing Pyramid: Strategy for test distribution.

**Layers:****1. Unit Tests (70%):**

- Test individual functions/classes
- Fast (milliseconds)
- Run on every commit
- Tools: JUnit, pytest, Jest

2. Integration Tests (20%):

- Test component interactions
- Medium speed (seconds)
- Database, API calls
- Tools: TestContainers, Postman

3. E2E Tests (10%):

- Test complete user flows
- Slow (minutes)
- UI automation
- Tools: Selenium, Cypress, Playwright

CI/CD integration:

- Unit: Always run
- Integration: Run on PR and main
- E2E: Run nightly or before deployment

Q22. What is test automation in CI/CD?

Answer:

Test Automation: Running tests automatically in pipeline.

Types:

1. Unit Tests:

```
- name: Run unit tests
  run: npm test
```

2. Integration Tests:

```
- name: Integration tests
  run: |
    docker-compose up -d
    npm run test:integration
    docker-compose down
```

3. Code Quality:

```
- name: Lint
  run: eslint .

- name: Code coverage
  run: jest --coverage
```

4. Security Scans:

- ```
- name: Security audit
 run: npm audit

- name: SAST scan
 run: sonar-scanner
```

#### 5. Performance Tests:

- Load testing (JMeter, K6)
- Benchmark tests

#### Best practices:

- Fast feedback (<10 min)
- Fail fast (stop on first failure)
- Parallel execution
- Clear failure messages

Q23. What is code coverage and why is it important?

**Answer:**

**Code Coverage:** Percentage of code executed by tests.

**Types:**

- **Line coverage:** % of lines executed
- **Branch coverage:** % of if/else paths taken
- **Function coverage:** % of functions called
- **Statement coverage:** % of statements executed

**Tools:**

- JavaScript: Istanbul, NYC
- Python: Coverage.py
- Java: JaCoCo
- Go: go test -cover

**In pipeline:**

- ```
- name: Test with coverage
  run: npm test -- --coverage

- name: Upload coverage
  uses: codecov/codecov-action@v3
```

```
- name: Enforce minimum
  run: |
    coverage=$(cat coverage.txt | grep total | awk '{print $4}')
    if [ ${coverage%}% -lt 80 ]; then
      echo "Coverage ${coverage} below 80%"
      exit 1
    fi
```

Best practices:

- Target: 70-90% (not 100%)
- Focus on critical paths
- Quality over quantity
- Combine with other metrics

Q24. What is static code analysis (SAST)?

Answer:

SAST (Static Application Security Testing): Analyze code without executing it.

What it finds:

- Security vulnerabilities
- Code smells
- Code duplication
- Complexity issues
- Style violations
- Potential bugs

Tools:

- **SonarQube:** Multi-language, comprehensive
- **ESLint:** JavaScript/TypeScript
- **Pylint:** Python
- **Checkstyle:** Java
- **Semgrep:** Pattern-based analysis

SonarQube in pipeline:

```
- name: SonarQube scan
  run: |
    sonar-scanner \
      -Dsonar.projectKey=myproject \
      -Dsonar.sources=src \
      -Dsonar.host.url=$SONAR_URL \
      -Dsonar.login=$SONAR_TOKEN
```

Quality gates:

- Minimum coverage: 80%
- No critical vulnerabilities
- Maximum technical debt: 5%
- Duplicated lines: ❤️ %

Benefits:

- Early bug detection
- Security issues found
- Enforce standards
- Reduce technical debt

Q25. What is dynamic analysis (DAST)?

Answer:

DAST (Dynamic Application Security Testing): Test running application.

vs SAST:

SAST	DAST
White box	Black box
Source code	Running app
Find potential issues	Find actual issues
Fast	Slower
False positives	More accurate

What DAST finds:

- SQL injection
- XSS (Cross-site scripting)
- Authentication issues
- Configuration errors
- Runtime vulnerabilities

Tools:

- **OWASP ZAP:** Open-source
- **Burp Suite:** Popular commercial
- **Acunetix:** Web vulnerability scanner
- **Veracode:** Enterprise platform

In pipeline:

```
- name: Deploy to test  
  run: ./deploy-test.sh
```

```

- name: DAST scan
  run: |
    zap-baseline.py -t https://test.myapp.com \
      -r zap-report.html

- name: Check results
  run: |
    if grep -q "High" zap-report.html; then
      echo "High severity issues found"
      exit 1
    fi

```

Best practice: Run on staging before production.

Q26. Explain smoke tests and health checks.

Answer:

Smoke Tests: Quick tests to verify basic functionality after deployment.

Purpose:

- Verify deployment succeeded
- Check critical paths
- Fast feedback (<5 min)

Examples:

```

# HTTP health check
curl -f https://api.example.com/health || exit 1

# Database connection
psql $DB_URL -c "SELECT 1" || exit 1

# Redis connection
redis-cli -h $REDIS_HOST ping || exit 1

# Critical endpoint
curl -f https://api.example.com/users/1 || exit 1

```

Health Check Endpoint:

```

app.get('/health', (req, res) => {
  const health = {
    status: 'healthy',
    timestamp: Date.now(),
    checks: {
      database: checkDatabase(),
      redis: checkRedis(),
    }
  }
  res.json(health)
})

```

```
    external_api: checkExternalAPI()
  }
};

const isHealthy = Object.values(health.checks)
  .every(check => check.status === 'ok');

res.status(isHealthy ? 200 : 503).json(health);
});
```

In pipeline:

```
- name: Deploy
  run: kubectl apply -f k8s/

- name: Wait for rollout
  run: kubectl rollout status deployment/myapp

- name: Smoke tests
  run: ./smoke-tests.sh

- name: Rollback on failure
  if: failure()
  run: kubectl rollout undo deployment/myapp
```

Q27. What is regression testing in CI/CD?

Answer:

Regression Testing: Verify existing functionality still works after changes.

Purpose:

- Catch breaking changes
- Ensure no features broken
- Maintain quality over time

Types:

1. Unit Regression:

- All unit tests
- Run on every commit

2. Integration Regression:

- Full test suite
- Run on PR merge

3. Visual Regression:

- Screenshot comparison
- Detect UI changes

4. Performance Regression:

- Benchmark tests
- Detect slowdowns

Visual regression example:

```
- name: Visual regression
  uses: percy/exec-action@v0.3.1
  with:
    command: npm run test:visual
  env:
    PERCY_TOKEN: ${{ secrets.PERCY_TOKEN }}
```

Tools:

- **Percy:** Visual testing
- **Chromatic:** Storybook visual testing
- **BackstopJS:** Screenshot comparison
- **Playwright:** E2E with screenshots

Best practices:

- Comprehensive test suite
- Fast execution (parallel)
- Clear failure reporting
- Automatic on all PRs

Q28. What is test parallelization?

Answer:

Test Parallelization: Run tests concurrently to reduce time.

Strategies:

1. File-level:

```
strategy:
  matrix:
    shard: [1, 2, 3, 4]
steps:
  - name: Run tests
    run: npm test --shard=${{ matrix.shard }}/4
```

2. Suite-level:

```
jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - run: npm run test:unit

  integration-tests:
    runs-on: ubuntu-latest
    steps:
      - run: npm run test:integration
```

3. Test-level (Pytest):

```
pytest -n 4 # 4 parallel processes
```

Benefits:

- **Faster feedback:** 10 min → 3 min
- **Better resource utilization**
- **Scalable:** Add more workers

Considerations:

- Test isolation required
- No shared state
- Database per worker
- Cost (more runners)

Example (GitHub Actions matrix):

```
strategy:
  matrix:
    node: [14, 16, 18]
    os: [ubuntu, windows, macos]
runs-on: ${{ matrix.os }}-latest
steps:
  - uses: actions/setup-node@v3
    with:
      node-version: ${{ matrix.node }}
  - run: npm test
```

Q29. What is contract testing?

Answer:

Contract Testing: Verify services communicate correctly without integration tests.

Problem: Microservices need to agree on API contracts.

Solution: Test consumer's expectations vs provider's implementation.

Types:

1. Consumer-Driven Contracts (CDC):

- Consumer defines expected contract
- Provider verifies it meets contract

2. Provider-Driven Contracts:

- Provider publishes spec (OpenAPI)
- Consumer tests against spec

Tool: Pact:

Consumer test:

```
const pact = new Pact({...});

it('gets user by ID', async () => {
  await pact.addInteraction({
    state: 'user 123 exists',
    uponReceiving: 'a request for user 123',
    withRequest: {
      method: 'GET',
      path: '/users/123'
    },
    willRespondWith: {
      status: 200,
      body: { id: 123, name: 'John' }
    }
  });

  const user = await getUser(123);
  expect(user.name).toBe('John');
});
```

Provider verification:

```
pact.verifyProvider({
  provider: 'UserService',
  pactUrls: ['pact-broker/contracts']
});
```

Benefits:

- Fast (no real services needed)

- Catch breaking changes early
- Clear contracts
- Independent deployment

Q30. What is test data management in CI/CD?

Answer:

Test Data Management: Handling data for automated tests.

Strategies:

1. Fixed Test Data:

```
-- seed.sql
INSERT INTO users VALUES (1, 'test@example.com', 'hash');
INSERT INTO products VALUES (1, 'Widget', 9.99);
```

2. Generated Data:

```
const user = faker.user({
  email: faker.internet.email(),
  name: faker.name.fullName()
});
```

3. Fixtures:

```
// fixtures/users.json
[
  { "id": 1, "email": "test1@example.com" },
  { "id": 2, "email": "test2@example.com" }
]
```

4. Test Containers:

```
const postgres = await new PostgreSQLContainer().start();
// Fresh DB for each test
```

In pipeline:

```
- name: Setup test DB
  run: |
    docker run -d -p 5432:5432 postgres:15
    npm run db:migrate
```

```
npm run db:seed

- name: Run tests
  run: npm test

- name: Cleanup
  run: docker stop postgres
```

Best practices:

- Isolated data per test
- Fast setup (<1 min)
- Realistic data
- No production data in tests
- Clean up after tests

Security & Compliance (Questions 31-40)

Q31. What is DevSecOps?

Answer:

DevSecOps: Integrating security practices into DevOps.

Shift Left Security: Find security issues early in development.

Security in CI/CD pipeline:**1. Source Code** (Commit):

- Pre-commit hooks
- Secrets scanning
- IDE security plugins

2. Build:

- Dependency vulnerability scanning
- SAST (static analysis)
- License compliance

3. Test:

- DAST (dynamic analysis)
- Security tests
- Penetration testing

4. Deploy:

- Container scanning
- Infrastructure security
- Configuration audit

5. Monitor:

- Runtime security
- Anomaly detection
- Incident response

Example pipeline with security:

```
stages:
  - lint
  - security-scan
  - build
  - test
  - security-test
  - deploy
  - monitor

security-scan:
  script:
    - trivy fs . # Dependency scan
    - semgrep --config=auto # SAST
    - gitleaks detect # Secrets

container-scan:
  script:
    - trivy image myapp:latest
    - docker scan myapp:latest
```

Q32. How do you handle secrets in CI/CD?

Answer:

Secret Management: Securely store and access sensitive data.

What are secrets?

- API keys
- Passwords
- SSH keys
- Certificates
- Tokens

Anti-patterns (DON'T):

- ✗ Hardcode in code
- ✗ Commit to Git
- ✗ Plain text in CI config
- ✗ Environment variables in Dockerfile

Best practices (DO):

1. Secret Management Tools:

- **HashiCorp Vault:** Enterprise secret management
- **AWS Secrets Manager:** AWS integration
- **Azure Key Vault:** Azure integration
- **Google Secret Manager:** GCP integration

2. CI/CD Platform Secrets:

```
# GitHub Actions
env:
  API_KEY: ${ secrets.API_KEY }

# GitLab CI
variables:
  API_KEY: $CI_JOB_TOKEN
```

3. Vault Integration:

```
- name: Get secrets
  run: |
    vault login -method=aws
    export DB_PASSWORD=$(vault kv get -field=password secret/db)
```

4. Encrypted files:

```
# Encrypt
gpg --encrypt --recipient "team@example.com" secrets.env

# Decrypt in pipeline
gpg --decrypt secrets.env.gpg > secrets.env
source secrets.env
```

5. Secrets scanning:

```
- name: Scan for secrets
  uses: trufflesecurity/trufflehog@main
  with:
    path: ./
```

Rotation: Regularly rotate secrets (30-90 days).

Q33. What is container security scanning?

Answer:

Container Scanning: Analyze Docker images for vulnerabilities.

What to scan:

- Base image vulnerabilities
- Dependency vulnerabilities
- Malware
- Misconfigurations
- Secrets in layers

Tools:

- **Trivy:** Fast, accurate, open-source
- **Snyk:** Developer-focused
- **Aqua Security:** Enterprise
- **Clair:** CoreOS project
- **Docker Scout:** Docker's scanner

Trivy in pipeline:

```
- name: Build image
  run: docker build -t myapp:$TAG .

- name: Scan image
  run: |
    trivy image \
      --severity HIGH,CRITICAL \
      --exit-code 1 \
      myapp:$TAG
```

Multi-stage scan:

```
# Scan dependencies before build
- name: Scan dependencies
  run: trivy fs --severity HIGH,CRITICAL .

# Scan final image
- name: Scan image
  run: trivy image myapp:$TAG

# Sign image if clean
- name: Sign image
  if: success()
  run: cosign sign myapp:$TAG
```

Best practices:

- Scan on every build

- Fail pipeline on HIGH/CRITICAL
- Use minimal base images (alpine, distroless)
- Scan both build-time and runtime
- Keep base images updated

Q34. What is compliance as code?

Answer:

Compliance as Code: Automate compliance checks using code.

Why?

- Manual audits are slow
- Consistent enforcement
- Continuous compliance
- Audit trail

Frameworks:

- **Open Policy Agent (OPA):** Policy engine
- **Checkov:** IaC security scanning
- **Inspec:** Compliance testing
- **Cloud Custodian:** Cloud compliance

OPA example (Kubernetes admission control):

```
# Policy: All containers must have resource limits
package kubernetes.admission

deny[msg] {
  input.request.kind.kind == "Pod"
  container := input.request.object.spec.containers[_]
  not container.resources.limits
  msg := sprintf("Container %v must have resource limits", [container.name])
}
```

Checkov for Terraform:

```
- name: Terraform security scan
  run: |
    checkov -d terraform/ \
      --framework terraform \
      --output junitxml > checkov-report.xml
```

Compliance checks:

- GDPR: Data encryption, access logs

- SOC 2: Audit logging, access control
- HIPAA: PHI protection, encryption
- PCI DSS: Payment data security

In pipeline:

```
compliance:
  script:
    - opa test policies/ # Test policies
    - conftest test k8s/ --policy policies/ # Validate K8s
    - checkov -d terraform/ # Scan IaC
```

Q35. Explain dependency vulnerability scanning.

Answer:

Dependency Scanning: Check third-party libraries for known vulnerabilities.

Why important?

- 80% of code is dependencies
- CVEs (Common Vulnerabilities and Exposures)
- Supply chain attacks
- License compliance

Tools by language:

- **npm:** npm audit, Snyk
- **Python:** safety, pip-audit
- **Java:** OWASP Dependency-Check
- **Go:** govulncheck
- **Ruby:** bundler-audit

In pipeline:**NPM:**

```
- name: Audit dependencies
  run: npm audit --audit-level=moderate

- name: Snyk scan
  run: snyk test --severity-threshold=high
```

Python:

```
- name: Safety check
  run: |
```

```
pip install safety
safety check --json > safety-report.json
```

GitHub Dependabot:

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10
```

Handling results:

```
- name: Check vulnerabilities
  run: |
    npm audit --json > audit.json
    HIGH=$(jq '.metadata.vulnerabilities.high' audit.json)
    if [ $HIGH -gt 0 ]; then
      echo "Found $HIGH high severity vulnerabilities"
      exit 1
    fi
```

Best practices:

- Scan on every build
- Auto-update minor versions
- Review major version updates
- Monitor continuously
- Have remediation process

Q36. What is SBOM (Software Bill of Materials)?

Answer:

SBOM: Complete inventory of software components and dependencies.

Why needed?

- Know what's in your software
- Vulnerability management
- License compliance
- Supply chain security
- Regulatory requirements

Formats:

- **SPDX:** Linux Foundation standard
- **CycloneDX:** OWASP standard
- **SWID:** ISO standard

Generate SBOM:**Syft (all languages):**

```
- name: Generate SBOM
  run: |
    syft packages dir:. -o spdx-json > sbom.json
    syft packages docker:myapp:latest -o cyclonedx > sbom.xml
```

npm:

```
npm sbom --sbom-format=cyclonedx > sbom.json
```

SBOM content:

```
{
  "bomFormat": "CycloneDX",
  "components": [
    {
      "type": "library",
      "name": "express",
      "version": "4.18.2",
      "licenses": ["MIT"],
      "purl": "pkg:npm/express@4.18.2"
    }
  ]
}
```

Use cases:

- Vulnerability tracking
- License auditing
- Procurement requirements
- Incident response

In pipeline:

```
- name: Generate and upload SBOM
  run: |
    syft packages . -o spdx-json > sbom.json
```

```
# Upload to artifact repository
curl -X POST -F "file=@sbom.json" $SBOM_REGISTRY
```

Q37. What is infrastructure security in CI/CD?

Answer:

Infrastructure Security: Secure the infrastructure running CI/CD.

Attack surfaces:

- CI/CD platform itself
- Build agents/runners
- Artifact repositories
- Secret stores
- Cloud resources

Security measures:

1. Access Control:

- RBAC (Role-Based Access Control)
- Least privilege principle
- MFA for admins

2. Network Security:

- Private runners
- VPC isolation
- Firewall rules
- No public endpoints

3. Runner Security:

```
# Use ephemeral runners
- name: Run in container
  container:
    image: node:18
  steps:
    - run: npm test
# Container destroyed after job
```

4. Audit Logging:

- Who did what and when
- Pipeline execution logs
- Access logs
- Change history

5. Secrets Isolation:

- Different secrets per environment
- Time-limited tokens
- Rotate regularly

6. Supply Chain:

- Pin action versions

```
# ✓ Good - pinned hash
- uses: actions/checkout@8e5e7e5ab8b370d6c329ec480221332ada57f0ab

# ✗ Bad - mutable tag
- uses: actions/checkout@v3
```

7. Code Signing:

```
- name: Sign artifacts
  run: cosign sign --key cosign.key myapp:$TAG
```

Compliance:

- SOC 2 Type II
- ISO 27001
- FedRAMP (government)

Q38. What is secrets rotation in CI/CD?

Answer:

Secrets Rotation: Regularly change secrets to limit exposure.

Why rotate?

- Limit blast radius if compromised
- Compliance requirements
- Security best practice
- Detect unauthorized access

Rotation frequency:

- **High risk:** 30 days (prod DB passwords)
- **Medium risk:** 90 days (API keys)
- **Low risk:** 180 days (dev credentials)
- **Immediate:** On suspected compromise

Automated rotation:

1. AWS Secrets Manager:

```
import boto3

def rotate_secret(event):
    client = boto3.client('secretsmanager')

    # Generate new password
    new_password = generate_password()

    # Update database
    update_database_password(new_password)

    # Update secret
    client.update_secret(
        SecretId=event['SecretId'],
        SecretString=json.dumps({'password': new_password})
    )
```

2. Vault dynamic secrets:

```
# Database credentials valid for 1 hour
vault read database/creds/myapp
```

3. Service account tokens:

```
# Kubernetes - auto-rotate tokens
apiVersion: v1
kind: ServiceAccount
metadata:
  name: myapp
automountServiceAccountToken: true
```

In pipeline:

```
- name: Fetch secrets
  run: |
    # Get current secrets from Vault
    vault kv get -field=api_key secret/myapp

- name: Deploy with secrets
  env:
    API_KEY: ${ secrets.API_KEY }
  run: ./deploy.sh
```

Best practices:

- Automate rotation
- Zero-downtime rotation
- Monitor for failures
- Alert on rotation issues
- Test rotation process

Q39. What is least privilege in CI/CD?

Answer:

Least Privilege: Grant minimum permissions needed to perform task.

Apply to:**1. CI/CD Platform Users:**

- Developers: Read pipelines, trigger builds
- DevOps: Full access
- QA: Deploy to test only

2. Pipeline Permissions:

```
# GitHub Actions
permissions:
  contents: read # Only read repo
  packages: write # Push images
  issues: none # No issue access
```

3. Service Accounts:

```
# Kubernetes - minimal permissions
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ci-deployer

---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "update", "patch"]
# Can only update deployments, nothing else
```

4. AWS IAM:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "ecr:PutImage",
      "ecr:BatchCheckLayerAvailability"
    ],
    "Resource": "arn:aws:ecr:us-east-1:123456789:repository/myapp"
  }]
}
```

5. Secrets Access:

```
# Only production deploy job can access prod secrets
deploy-prod:
  environment: production
  secrets:
    - PROD_DB_PASSWORD
```

Enforce:

- RBAC (Role-Based Access Control)
- Policy as Code (OPA)
- Regular audits
- Remove unused permissions

Benefits:

- Limit damage from compromised credentials
- Compliance (SOC 2, ISO 27001)
- Clear responsibility
- Audit trail

Q40. What is pipeline security hardening?

Answer:

Pipeline Hardening: Secure the CI/CD pipeline itself.

Security measures:

1. Input Validation:

```
# Validate branch names
- name: Check branch
  run: |
    if [[ ! "$GITHUB_REF" =~ ^refs/heads/(main|release/.*)$ ]]; then
```

```
    echo "Invalid branch"
    exit 1
  fi
```

2. Prevent Code Injection:

```
# ✗ BAD - Command injection risk
- run: echo ${ { github.event.issue.title } }

# ✓ GOOD - Use environment variable
- env:
  TITLE: ${ { github.event.issue.title } }
  run: echo "$TITLE"
```

3. Pin Dependencies:

```
# Pin action versions with SHA
- uses: actions/checkout@8e5e7e5ab8b370d6c329ec480221332ada57f0ab

# Pin base images with digest
FROM node:18@sha256:a6385a...
```

4. Immutable Infrastructure:

```
# Don't modify runners, use fresh ones
runs-on: ubuntu-latest # Fresh runner each time
```

5. Audit Logging:

```
- name: Log deployment
  run: |
    echo "Deployed by: $GITHUB_ACTOR"
    echo "Commit: $GITHUB_SHA"
    echo "Time: $(date -u +%Y-%m-%dT%H:%M:%SZ)"
    # Send to SIEM
```

6. Approval Gates:

```
deploy-prod:
  needs: [build, test]
  environment:
    name: production
    # Requires manual approval
```

7. Network Isolation:

```
# Self-hosted runners in private network
runs-on: [self-hosted, private]
```

8. Secrets Masking:

```
- name: Mask secrets
  run: |
    echo "::add-mask::$SECRET_VALUE"
    echo "Deploying with secret: $SECRET_VALUE"
    # Output: Deploying with secret: ***
```

Checklist:

- ☒ All secrets in vault, not code
- ☒ Least privilege permissions
- ☒ Approval for production
- ☒ Audit all pipeline changes
- ☒ Scan for vulnerabilities
- ☒ Pin all dependencies
- ☒ Network isolation
- ☒ Monitoring and alerting

Monitoring & Operations (Questions 41-50)

Q41. What is observability in CI/CD?

Answer:

Observability: Ability to understand system's internal state from external outputs.

Three Pillars:

1. Metrics:

- Pipeline success/failure rate
- Build duration
- Deployment frequency
- Lead time for changes
- Mean time to recovery (MTTR)

2. Logs:

- Build logs
- Test output

- Deployment logs
- Error messages

3. Traces:

- Pipeline execution flow
- Dependency chains
- Performance bottlenecks

DORA Metrics (DevOps Research and Assessment):

- **Deployment Frequency:** How often you deploy
- **Lead Time:** Commit to production time
- **Change Failure Rate:** % of deployments causing issues
- **Mean Time to Recovery:** Time to fix production issues

Tools:

- **Prometheus + Grafana:** Metrics
- **ELK Stack:** Logs (Elasticsearch, Logstash, Kibana)
- **Datadog:** All-in-one
- **New Relic:** APM
- **Jaeger:** Distributed tracing

Pipeline metrics example:

```
- name: Record metrics
  run: |
    # Send to Prometheus pushgateway
    echo "pipeline_duration_seconds $(date +%s)" | \
    curl --data-binary @- http://pushgateway:9091/metrics/job/ci
```

Q42. How do you monitor CI/CD pipelines?

Answer:

Pipeline Monitoring: Track pipeline health and performance.

What to monitor:

1. Success Rate:

```
Success Rate = Successful Builds / Total Builds
Target: >95%
```

2. Build Duration:

Track p50, p95, p99 percentiles
Target: <10 minutes for fast feedback

3. Flaky Tests:

Tests that pass/fail inconsistently
Track failure patterns

4. Queue Time:

Time waiting for available runner
High = need more runners

5. Resource Usage:

CPU, memory, disk usage
Optimize resource allocation

Dashboards:

```
# Grafana dashboard
- Build success rate (last 7 days)
- Average build time trend
- Failed builds by stage
- Top 10 slowest tests
- Runner utilization
- Deployment frequency
```

Alerts:

```
# Alert rules
- name: HighFailureRate
  condition: success_rate < 80%
  action: notify_team

- name: SlowBuilds
  condition: avg_duration > 20min
  action: investigate

- name: DeploymentFailed
  condition: prod_deployment = failed
```

```
severity: critical  
action: page_oncall
```

Log aggregation:

```
- name: Send logs to ELK  
  run: |  
    # Ship logs to Elasticsearch  
    cat build.log | \  
    filebeat -e \  
    -E output.elasticsearch.hosts=["elk:9200"]
```

Q43. What are deployment strategies and when to use each?

Answer:

Deployment Strategies: Methods to release new versions.

1. Recreate (Big Bang):

- Stop all old
- Deploy all new
- **Downtime:** Yes
- **Use:** Development, maintenance windows

2. Rolling:

- Gradually replace instances
- **Downtime:** No
- **Use:** Standard deployments

3. Blue-Green:

- Two identical environments
- Switch traffic instantly
- **Downtime:** No
- **Cost:** High (2x infrastructure)
- **Use:** Mission-critical apps

4. Canary:

- Release to small % first
- **Downtime:** No
- **Risk:** Low
- **Use:** High-risk changes

5. A/B Testing:

- Different versions to different users

- **Purpose:** Feature comparison
- **Use:** Testing new features

6. Shadow:

- Production traffic duplicated to new version
- **Risk:** None (results not used)
- **Use:** Testing without impact

Decision matrix:

```
Low risk + small app → Rolling
High risk → Canary
Zero downtime required → Blue-Green
Testing features → A/B
Mission critical → Blue-Green + Canary
Cost sensitive → Rolling
```

Kubernetes strategies:

```
# Rolling (default)
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0

# Recreate
strategy:
  type: Recreate
```

Q44. What is GitOps?

Answer:

GitOps: Using Git as single source of truth for infrastructure and applications.

Principles:

1. **Declarative:** Desired state in Git
2. **Versioned:** All changes in Git history
3. **Automated:** Changes automatically applied
4. **Auditable:** Complete audit trail

Traditional vs GitOps:

```
Traditional:
Developer → CI → Manual kubectl → Cluster
```

```
GitOps:  
Developer → Git commit → GitOps operator → Cluster
```

Tools:

- **ArgoCD**: Kubernetes GitOps
- **Flux**: CNCF project
- **Jenkins X**: Cloud-native CI/CD

ArgoCD example:

```
# Git repository structure  
repo/  
├── k8s/  
│   ├── deployment.yaml  
│   ├── service.yaml  
│   └── ingress.yaml  
  
# ArgoCD watches this repo  
# Any change automatically deployed
```

Benefits:

- Git history = deployment history
- Easy rollback (git revert)
- Disaster recovery (restore from Git)
- Multi-cluster sync
- Declarative infrastructure

Workflow:

1. Developer updates k8s manifests in Git
2. Pull request and review
3. Merge to main
4. ArgoCD detects change
5. ArgoCD applies to cluster
6. Monitors and syncs continuously

Application manifest:

```
apiVersion: argoproj.io/v1alpha1  
kind: Application  
metadata:  
  name: myapp  
spec:  
  source:
```

```

repoURL: https://github.com/org/repo
path: k8s
targetRevision: main
destination:
  server: https://kubernetes.default.svc
  namespace: production
syncPolicy:
  automated:
    prune: true
    selfHeal: true

```

Q45. Explain rollback strategies.

Answer:

Rollback: Revert to previous working version.

Methods:

1. Kubernetes Rollout Undo:

```

# Check history
kubectl rollout history deployment/myapp

# Rollback to previous
kubectl rollout undo deployment/myapp

# Rollback to specific revision
kubectl rollout undo deployment/myapp --to-revision=3

```

2. Git Revert (GitOps):

```

git revert HEAD
git push
# GitOps tool auto-deploys previous version

```

3. Blue-Green Switch:

```

# Switch back to blue environment
kubectl patch service myapp -p '{"spec":{"selector":{"version":"blue"}}}'

```

4. Redeploy Previous Artifact:

```

- name: Rollback
  run: |

```

```
# Deploy previous Docker image
docker pull myapp:previous-tag
docker tag myapp:previous-tag myapp:latest
kubectl set image deployment/myapp myapp=myapp:previous-tag
```

5. Database Rollback:

```
# More complex - need migration down
npm run migrate:down
# Or restore from backup
```

Automated rollback:

```
- name: Deploy
  run: kubectl apply -f k8s/

- name: Smoke tests
  run: ./smoke-tests.sh
  timeout-minutes: 5

- name: Monitor metrics
  run: ./check-error-rate.sh
  timeout-minutes: 10

- name: Auto-rollback on failure
  if: failure()
  run: |
    echo "Deployment failed, rolling back"
    kubectl rollout undo deployment/myapp
    # Alert team
```

Best practices:

- Test rollback procedure regularly
- Keep previous version ready
- Database changes backward compatible
- Monitor after rollback
- Document rollback process
- Automate where possible

Q46. What is feature flag/toggle?

Answer:

Feature Flag: Control feature availability without deploying code.

Use cases:

1. Progressive Rollout:

```
if (featureFlag.isEnabled('new-checkout', userId)) {  
  return newCheckout();  
} else {  
  return oldCheckout();  
}
```

2. A/B Testing:

```
const variant = featureFlag.getVariant('payment-button', userId);  
// 50% see blue button, 50% see green
```

3. Kill Switch:

```
if (featureFlag.isEnabled('recommendation-engine')) {  
  // Enable/disable instantly without deploy  
}
```

4. Trunk-Based Development:

```
// Deploy incomplete feature behind flag  
if (featureFlag.isEnabled('new-feature')) {  
  // Work in progress  
}
```

Tools:

- **LaunchDarkly**: Enterprise
- **Unleash**: Open-source
- **Split**: A/B testing focused
- **ConfigCat**: Developer-friendly

Implementation:

```
# In pipeline - deploy with flag OFF  
- name: Deploy with feature flag  
  run: |  
    kubectl set env deployment/myapp \  
      FEATURE_NEW_UI=false  
  
# Later, enable via feature flag service  
# No deployment needed!
```

Benefits:

- Decouple deploy from release
- Instant rollback (toggle off)
- Test in production safely
- Gradual rollout
- A/B testing

Best practices:

- Clean up old flags
- Flag expiration dates
- Monitor flag usage
- Document flags
- Don't overuse (tech debt)

Q47. What is chaos engineering in CI/CD?

Answer:

Chaos Engineering: Deliberately inject failures to test resilience.

Principles:

1. Define steady state
2. Hypothesize steady state continues
3. Inject real-world failures
4. Try to disprove hypothesis

Types of chaos:**1. Infrastructure:**

- Kill pods/containers
- Network latency
- CPU/Memory stress
- Disk full

2. Application:

- Simulate exceptions
- Slow responses
- Invalid data

3. Dependencies:

- Database failures
- External API timeouts
- Cache misses

Tools:

- **Chaos Monkey:** Netflix, kills instances
- **Litmus:** Kubernetes chaos
- **Gremlin:** Failure-as-a-Service
- **Chaos Mesh:** CNCF project

In pipeline:

```
chaos-test:
  stage: test
  script:
    # Deploy to test environment
    - kubectl apply -f k8s/

    # Run chaos experiments
    - litmus install
    - kubectl apply -f chaos/pod-delete.yaml

    # Verify system recovered
    - ./verify-resilience.sh
```

Example experiment (Kubernetes):

```
apiVersion: litmuschaos.io/v1alpha1
kind: ChaosEngine
metadata:
  name: pod-delete
spec:
  experiments:
    - name: pod-delete
      spec:
        components:
          env:
            - name: TOTAL_CHAOS_DURATION
              value: '60'
            - name: CHAOS_INTERVAL
              value: '10'
```

Benefits:

- Find weaknesses before production
- Improve resilience
- Test monitoring/alerting
- Build confidence

Start small:

- Non-production first
- Simple experiments

- Increase complexity gradually
- Have rollback ready

Q48. What is continuous feedback and monitoring?

Answer:

Continuous Feedback: Ongoing monitoring and feedback loops.

Feedback loops:

1. Development (seconds):

- IDE errors
- Linting
- Unit tests

2. Pre-commit (seconds):

- Pre-commit hooks
- Local tests

```
# .git/hooks/pre-commit
npm run lint
npm run test:unit
```

3. CI Pipeline (minutes):

- Build success/failure
- Test results
- Code quality

4. Staging (hours):

- Integration tests
- Performance tests
- Security scans

5. Production (continuous):

- Application metrics
- User behavior
- Error rates
- Business metrics

Feedback channels:

1. Pull Request:

```
- name: Comment on PR
  uses: actions/github-script@v6
  with:
    script: |
      github.rest.issues.createComment({
        issue_number: context.issue.number,
        body: '✅ Tests passed! Coverage: 85%'
      })
```

2. Slack/Teams:

```
- name: Notify team
  run: |
    curl -X POST $SLACK_WEBHOOK \
      -d '{"text":"Deploy to prod succeeded"}
```

3. Email:

```
- name: Email on failure
  if: failure()
  run: |
    echo "Build failed" | \
      mail -s "CI Failed" team@example.com
```

4. Dashboards:

- Real-time build status
- Deployment frequency
- Error rates
- Performance metrics

Monitoring in production:

```
- name: Deploy
  run: kubectl apply -f k8s/

- name: Monitor for 5 minutes
  run: |
    for i in {1..30}; do
      error_rate=$(curl -s metrics/error-rate)
      if [ $error_rate -gt 5 ]; then
        echo "Error rate too high: $error_rate%"
        kubectl rollout undo deployment/myapp
        exit 1
      fi
```

```
    sleep 10
done
```

Key metrics:

- Build duration trend
- Test success rate
- Deployment frequency
- MTTR (Mean Time To Recovery)
- Customer satisfaction

Q49. What are pipeline optimization techniques?

Answer:

Pipeline Optimization: Make pipelines faster and more efficient.

Techniques:**1. Caching:**

```
# Cache dependencies
- name: Cache npm
  uses: actions/cache@v3
  with:
    path: ~/.npm
    key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
```

Speeds up: 5 min → 30 sec

2. Parallel Execution:

```
jobs:
  test-unit:
    runs-on: ubuntu-latest
    steps: [run unit tests]

  test-integration:
    runs-on: ubuntu-latest
    steps: [run integration tests]

  test-e2e:
    runs-on: ubuntu-latest
    steps: [run e2e tests]

# All run simultaneously
```

3. Docker Layer Caching:

```
# Order matters - stable things first
FROM node:18
WORKDIR /app

# These rarely change - cached
COPY package*.json ./
RUN npm ci

# These change often - not cached
COPY . .
RUN npm run build
```

4. Incremental Builds:

```
- name: Check changed files
  id: changes
  run: |
    git diff --name-only HEAD~1

- name: Build frontend
  if: contains(steps.changes.outputs.files, 'frontend/')
  run: npm run build:frontend

- name: Build backend
  if: contains(steps.changes.outputs.files, 'backend/')
  run: npm run build:backend
```

5. Test Sharding:

```
strategy:
  matrix:
    shard: [1, 2, 3, 4]
steps:
  - run: npm test -- --shard=${{ matrix.shard }}/4

# 20 min → 5 min
```

6. Fail Fast:

```
- name: Lint (fast)
  run: npm run lint

- name: Type check (fast)
  run: npm run typecheck

- name: Unit tests (medium)
```

```
run: npm test

- name: E2E tests (slow)
  run: npm run test:e2e
```

7. Selective Testing:

```
- name: Run affected tests
  run: |
    # Only test changed modules
    nx affected:test --base=HEAD~1
```

8. Build Artifact Reuse:

```
build:
  - run: npm run build
  - uses: actions/upload-artifact@v3
    with:
      name: dist
      path: dist/

deploy:
  needs: build
  - uses: actions/download-artifact@v3
  - run: ./deploy.sh
```

Impact:

- Before: 30 min pipeline
- After optimizations: 8 min pipeline
- Developer productivity 
- Deployment frequency 

Q50. What are best practices for CI/CD pipelines?

Answer:

CI/CD Best Practices:

1. Speed:

- ☒ Keep builds under 10 minutes
- ☒ Fast feedback loop
- ☒ Parallel execution
- ☒ Cache dependencies

2. Reliability:

- ☒ Fix broken builds immediately
- ☒ Flaky test elimination
- ☒ Retry transient failures
- ☒ Monitor pipeline health

3. Security:

- ☒ Secrets in vault, not code
- ☒ Scan for vulnerabilities
- ☒ Least privilege access
- ☒ Sign artifacts

4. Testing:

- ☒ Comprehensive test suite
- ☒ Test pyramid (70% unit, 20% integration, 10% E2E)
- ☒ Security tests (SAST/DAST)
- ☒ Performance tests

5. Deployment:

- ☒ Automated deployments
- ☒ Blue-green or canary
- ☒ Easy rollback
- ☒ Smoke tests after deploy

6. Observability:

- ☒ Log everything
- ☒ Monitor metrics
- ☒ Alert on failures
- ☒ Track DORA metrics

7. Infrastructure:

- ☒ Infrastructure as Code
- ☒ Immutable infrastructure
- ☒ Environment parity
- ☒ Containerization

8. Process:

- ☒ Trunk-based development
- ☒ Small, frequent commits
- ☒ Code review on all changes
- ☒ Documentation

9. Culture:

- ☒ Shared responsibility
- ☒ Blameless post-mortems

- ☒ Continuous improvement
- ☒ Automation mindset

10. Compliance:

- ☒ Audit trail
- ☒ Compliance checks automated
- ☒ Data protection
- ☒ Change management

Pipeline template:

```
name: Production Pipeline

on: [push, pull_request]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Lint
        run: npm run lint

  test:
    needs: lint
    strategy:
      matrix:
        shard: [1, 2, 3, 4]
    steps:
      - name: Test
        run: npm test --shard=${{ matrix.shard }}/4

  security:
    needs: lint
    steps:
      - name: Security scan
        run: |
          npm audit
          trivy fs .

  build:
    needs: [test, security]
    steps:
      - name: Build
        run: npm run build
      - name: Upload artifact
        uses: actions/upload-artifact@v3

  deploy-staging:
    needs: build
    environment: staging
```

```
steps:
  - name: Deploy
    run: ./deploy.sh staging
  - name: Smoke tests
    run: ./smoke-tests.sh

deploy-prod:
  needs: deploy-staging
  environment: production
  steps:
    - name: Deploy
      run: ./deploy.sh production
    - name: Monitor
      run: ./monitor-deployment.sh
```

Continuous improvement:

- Review metrics weekly
- Identify bottlenecks
- Experiment with optimizations
- Learn from failures
- Share knowledge

Quick Reference

Common CI/CD Tools:

- **CI/CD:** Jenkins, GitLab CI, GitHub Actions, CircleCI
- **Containers:** Docker, Kubernetes, Helm
- **IaC:** Terraform, CloudFormation, Ansible
- **Security:** Trivy, Snyk, SonarQube
- **Monitoring:** Prometheus, Grafana, Datadog

Key Metrics (DORA):

1. Deployment Frequency
2. Lead Time for Changes
3. Change Failure Rate
4. Mean Time to Recovery

Deployment Strategies:

- Rolling: Gradual replacement
- Blue-Green: Instant switch
- Canary: Gradual traffic shift
- Recreate: Replace all at once

Security Checklist:

- ☒ Secrets management

- ☒ Vulnerability scanning
 - ☒ Least privilege
 - ☒ Audit logging
 - ☒ Compliance checks
-

Good luck with your CI/CD interview! 🚀